

Predicting Billboard #1 Hits

Authors: May Eindra Tet Toe, Anastasia Tountas, Tran Anh Thu Phung, Harry Nguyen

Summary

The objective of this analysis is to use publicly available song information from the Billboard Hot 100 song rankings to investigate the question:

Given various metrics about a song (energy, beats per minute, etc.), how many weeks, if any, will a song remain at the top position in the Billboard Hot 100 rankings? Which features contribute most to our model's predictions?

Introduction

The magazine “Billboard” is a leading source of information on trends in music popularity and innovation. Following the launch of the magazine in 1958, weekly rankings of the top 100 songs in the US have been made publicly available. (Billboard Staff 2011) The dataset that we have selected is open-source Tidy Tuesday (Data Science Learning Community 2025) dataset which documents these billboard rankings from August 4th 1958, until January 11th 2025. It provides features such as lyrics, song-structure, and featured_artists for each song in the ranking. These features can then be used for song popularity predictions or music trend forecasting. In this analysis, we use linear regression to predict the number of weeks a song remains at the #1 spot in the rankings, and assess our model's performance using RMSE, MAE and R^2 . Even though our results show that our regressor has a weak performance, information regarding feature importances in model decision-making remain useful in determining a song's overall popularity.

Methods and Results

Overview of our Methodology

The Billboard Hot 100 dataset provides us with two datasets: `billboard.csv`, which contains one row per song that reached #1 along with it's features, and `topics.csv`, a reference list of lyrical theme labels (`lyrical_topics`). Our team decided to combine the two datasets to `billboard_clean` in order to create more comprehensive data to better equip our model. Here is an overview of our data analysis methodology:

1. **Reading & Wrangling:** read and wrangle the dataset into one tidy combined dataset
2. **Exploratory Data Analysis:** explore the data to understand the distribution of features and their relationships with the target variable
3. **Feature Transformations & Selection:** applied `log1p()` to skewed variables, one-hot encoded `cdr_genre`, and collapsed `simplified_key` into three categories. Then, manually dropped irrelevant columns and used `nearZeroVar()` to remove predictors with near-zero variance.
4. **Model Building & Evaluation:** split the data 70/30, fit an ordinary least squares model using a `tidymodels` workflow. Then, evaluate the model using a repeated 5-fold cross-validation and test set metrics (RMSE, R^2 , MAE).
5. **Results and Conclusion:** summarize our findings, discuss the implications of our results, and suggest potential future work or improvements to our analysis

Load libraries and set seed

```
library(tidyverse)
library(tidyuesdayR)
library(knitr)
library(dplyr)
library(caret)
library(tidymodels)
```

```
set.seed(123)
```

Reading & Wrangling our dataset from the web into R

- The two datasets were joined using a `left_join()` on `lyrical_topic` (billboard) and `lyrical_topics` (topics).
- **Identifier columns** (`song`, `artist`, `date`) were removed as they are not predictive features.
- `non_consecutive` was removed to prevent data leakage — it can only be determined after a song has completed its full chart run.
- `rating_1`, `rating_2`, `rating_3` were removed in favour of `overall_rating` (their mean) to avoid multicollinearity.
- **Free-text columns** were removed to avoid the high dimensionality and noise associated with Natural Language Processing: `lyrics`, `u_s_artwork`, `sound_effects`, `song_structure`, `featured_artists`, `songwriters`, `songwriters_w_o_interpolation_sample_credits`, `producers`, `double_a_side`, and `talent_contestant`.
- **High-cardinality columns** removed because they generalise poorly: `label`, `parent_label`, and `artist_place_of_origin`.
- **Redundant columns** were removed: `keys` is replaced by `simplified_key`; `cdr_style` and `discogs_style` are replaced by `cdr_genre` and `discogs_genre` respectively.
- `discogs_genre` was removed in favour of `cdr_genre` because `cdr` has more reliable class definitions, thus improving internal validity.
- **Ambiguous columns** were removed: `featured_in_a_then_contemporary_play`, `featured_in_a_then_contemporary_tv_show`, and `featured_in_a_then_contemporary_t_v_show`.
- All remaining character columns were converted to factors using `as.factor()`, and rows with any missing values were dropped with `drop_na()`.

```
# Load the dataset
```

```
tuesdata <- tidyuesdayR::tt_load(2025, week = 34)
```

```
## ---- Compiling #TidyTuesday Information for 2025-08-26 ----
```

```
## --- There are 2 files available ---
```

```
##
```

```
##
```

```
## -- Downloading files -----
```

```
##
```

```
## 1 of 2: "billboard.csv"
```

```
## 2 of 2: "topics.csv"
```

```
# Extract the datasets
```

```
billboard <- tuesdata$billboard
```

```
topics <- tuesdata$topics
```

```
# Join the two datasets
```

```
billboard_joined <- billboard |>
```

```
  left_join(topics, by = c("lyrical_topic" = "lyrical_topics"))
```

```
billboard_clean <- billboard_joined |>
```

```
# Remove identifier columns
```

```

select(-c(song, artist, date)) |>
# Remove data-leakage column
select(-non_consecutive) |>
# Remove individual judge scores to avoid multicollinearity
select(-c(rating_1, rating_2, rating_3)) |>
# Remove free-text columns
select(-c(
  lyrics,
  u_s_artwork,
  sound_effects,
  song_structure,
  featured_artists,
  songwriters,
  songwriters_w_o_interpolation_sample_credits,
  producers,
  double_a_side,
  talent_contestant
)) |>
# Remove high-cardinality columns
select(-c(label, parent_label, artist_place_of_origin)) |>
# Remove redundant columns and discogs_genre
select(-c(cdr_style, discogs_style, keys, discogs_genre)) |>
# Remove ambiguous columns
select(-c(
  featured_in_a_then_contemporary_play,
  featured_in_a_then_contemporary_film,
  featured_in_a_then_contemporary_t_v_show
)) |>
# Convert remaining character columns to factors
mutate(across(where(is.character), as.factor)) |>
drop_na()

```

Exploratory Data Analysis

First, we find the summary statistics for our target variable, `weeks_at_number_one`, and our key numeric predictors: `overall_rating`, `bpm`, `energy`, `danceability`, `happiness`, `acousticness`, `loudness_d_b`, `front_person_age`, and `length_sec`.

Table 1: Summary statistics

Variable	Mean	SD	Min	Median	Max
acousticness	31.51	27.66	0	23.00	98
bpm	115.50	23.69	16	115.00	176
danceability	62.45	15.21	14	64.00	98
energy	59.81	19.62	3	60.50	98
front_person_age	27.97	6.51	11	27.50	62
happiness	62.60	24.96	4	68.00	99
length_sec	222.88	50.12	96	224.00	515
loudness_d_b	-9.02	3.40	-23	-9.00	-1
overall_rating	6.18	1.86	1	6.33	10
weeks_at_number_one	2.91	2.49	1	2.00	16

Then, we examine the distribution of our target variable, `weeks_at_number_one`, to check for skewness.

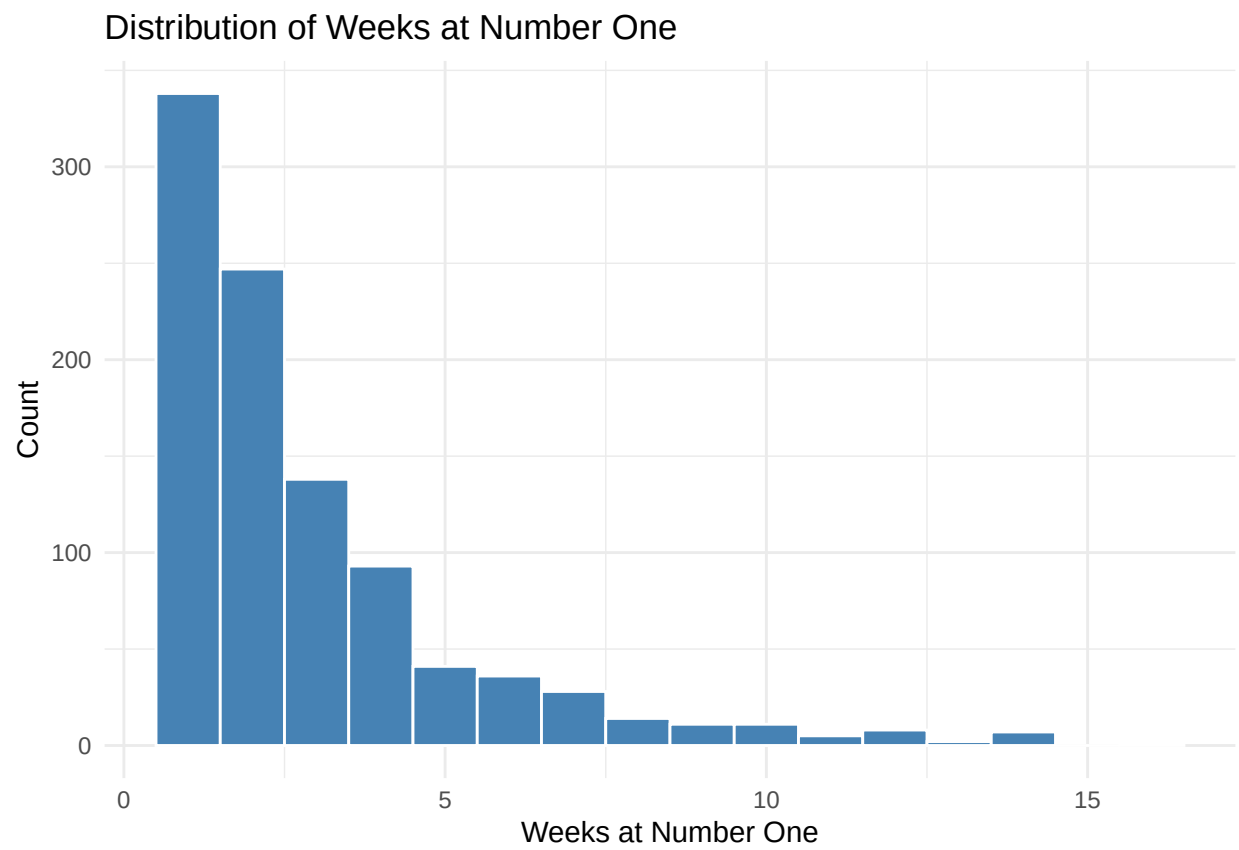


Figure 1: Distribution of weeks at number one.

The distribution is heavily right-skewed. The majority of songs spent only 1 to 3 weeks at #1, with counts dropping off sharply afterwards. A small number of songs stayed for 10 or more weeks, forming a long right tail.

Next, we examine the distributions of the five Spotify audio features, to identify any skewness that may require transformation before modelling.

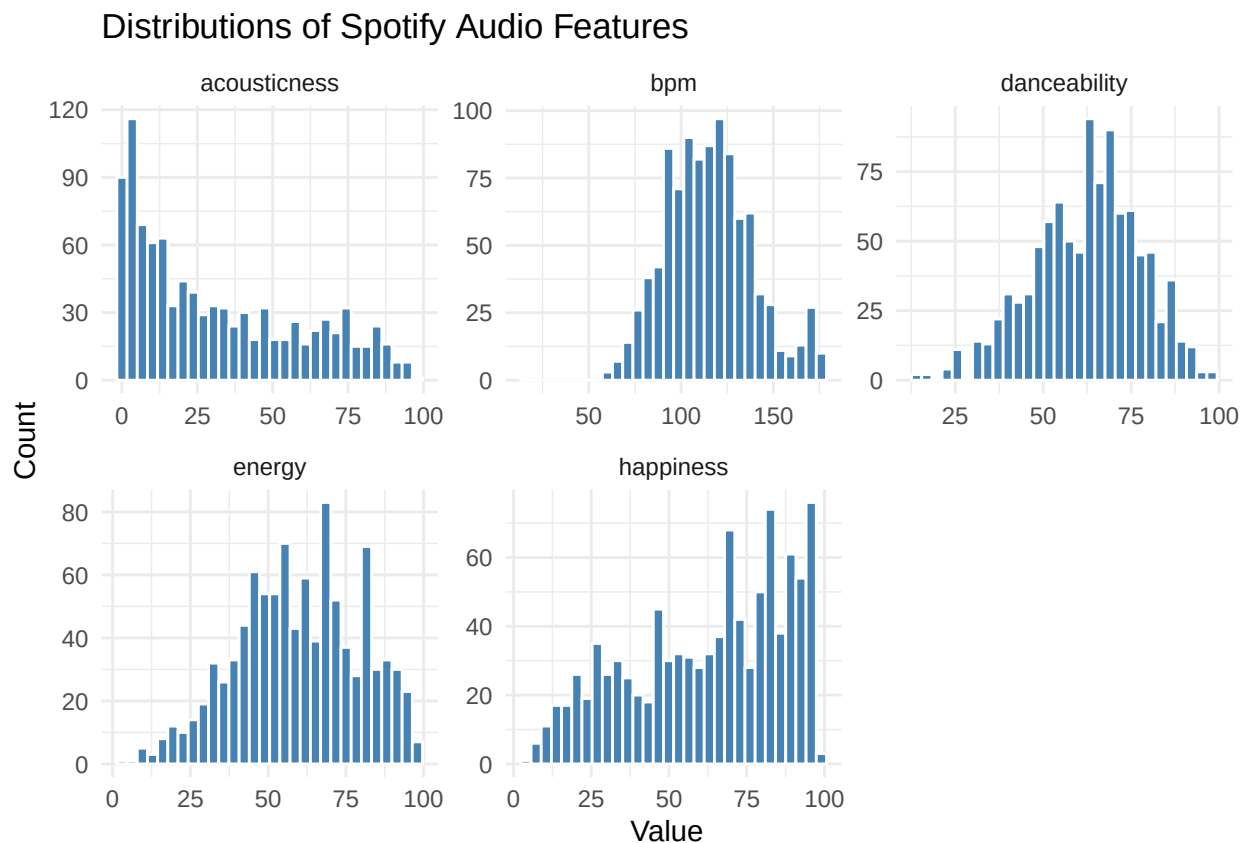


Figure 2: Distributions of the five Spotify audio features.

acousticness is heavily right-skewed, with most songs clustered near zero. **happiness** shows a bimodal pattern with peaks around 30 and around 80. **energy** is very slightly left-skewed, while **bpm** and **danceability** are roughly normal.

Finally, we compare average **weeks_at_number_one** by genre to see whether certain genres are associated with longer chart runs.

Electronic/Dance;Latin seems to have the highest average (~10 weeks). Pop;Reggae and Folk/Country;March also rank highly. Mainstream genres like Pop and Rock sit in the middle of the range. At the lower end, we can find niche genres like Electronic/Dance;Funk/Soul, averaging to 1 week.

Feature Transformations

Based on the Exploratory Data Analysis, two variables require transformation before modelling:

- **weeks_at_number_one** is heavily right-skewed
- **acousticness** is also heavily right-skewed

Therefore, we apply log transformation to these three variables to compresses the tail and produce a more symmetric target distribution. We will be using `log1p()` instead of `log()` to handle any zero values in the

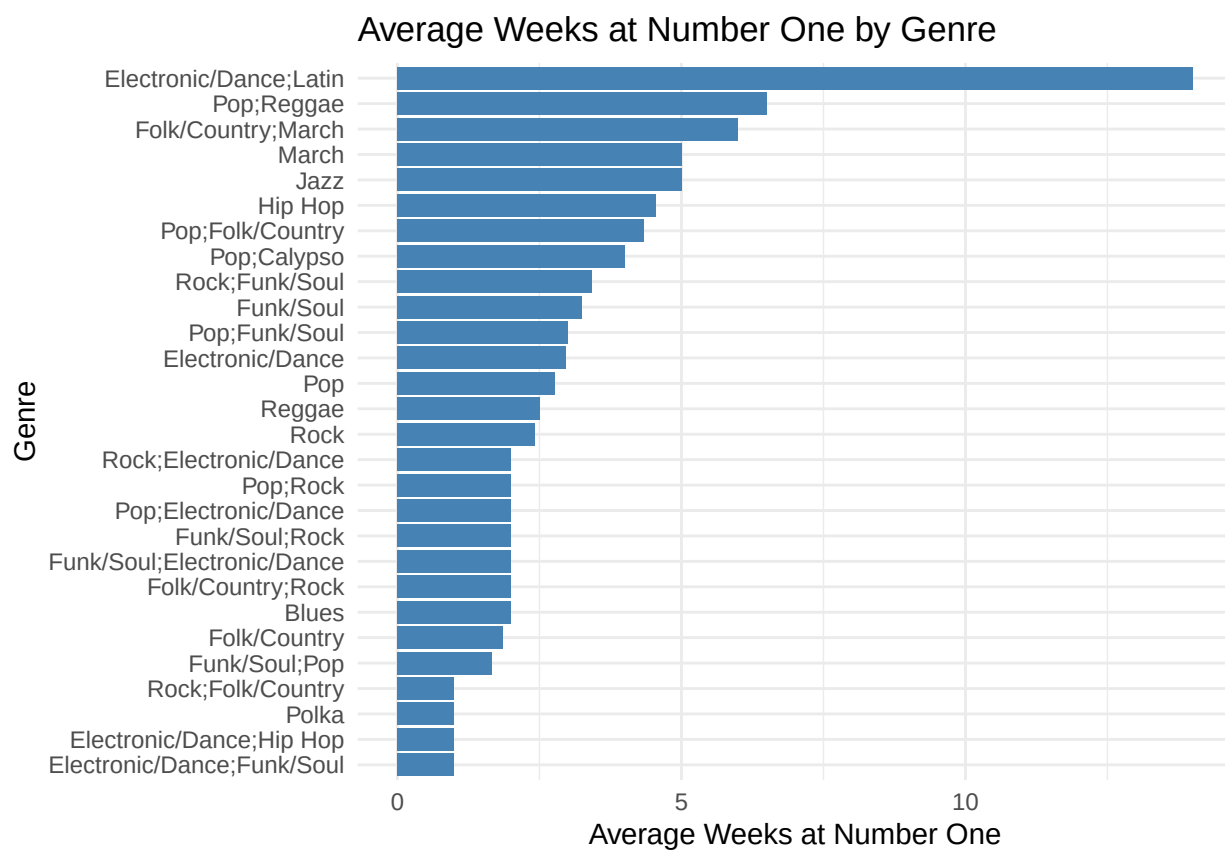


Figure 3: Average weeks at number one by genre.

data.

```
billboard_log_transformed <- billboard_clean |>
  mutate(
    log_weeks_at_number_one = log1p(weeks_at_number_one),
    log_acousticness = log1p(acousticness)
  )
```

Moreover, several songs in the dataset were associated with more than one musical genre (for example, Electronic/Dance; Latin or Pop; Funk/Soul). Treating these combined labels as single categorical values would incorrectly imply that each combination represents a distinct genre and would lead to sparse, poorly interpretable factor levels. To address this, the `cdr_genre` variable was processed as a multi-label feature. Individual genre were first separated into their component parts and then encoded as binary indicator variables using one-hot encoding.

```
billboard_genres <- billboard_log_transformed |>
  separate_rows(cdr_genre, sep = ";") |>
  mutate (cdr_genre = str_trim(cdr_genre))

billboard_genres_wide <- billboard_genres |>
  mutate(value = 1) |>
  pivot_wider(
    names_from = cdr_genre,
    values_from = value,
    values_fill = 0
  )
```

The `simplified_key` variable originally contained 25 levels representing individual musical keys in standard notation (e.g. C, Am, Bbm). Retaining all 25 levels would generate an excessive number of dummy variable during modelling, introducing sparsity and increasing the risk of overfitting. To address this, the keys were collapsed into three musically meaningful categories: Major (e.g. C, Ab, F), Minor (e.g. Am, Bbm, Em), and Multiple Keys.

```
billboard_simplify_key <- billboard_genres_wide |>
  mutate(
    simplified_key = case_when(
      simplified_key == "Multiple Keys" ~ "Multiple Keys",
      str_ends(simplified_key, "m") ~ "Minor",
      TRUE ~ "Major"
    ) |> as.factor()
  )

billboard_model_final <- billboard_simplify_key |>
  select(-weeks_at_number_one, -acousticness)
```

Feature Selection

Drop Redundant Columns Columns were dropped on conceptual grounds across five categories:

- Demographic identity columns encoding the race and gender of artists, songwriters, and producers.
- Artist structure indicators, songwriter/producer role overlap columns.
- Timing-derived columns.
- Lyric redundant content.
- External content.

```
columns_to_drop <- c(
  # Demographic identity
```

```

"artist_male", "artist_white", "artist_black", "songwriter_male", "songwriter_white", "producer_male",
# Artist structure
"artist_structure", "group_named_after_non_lead_singer",
# Songwriter/producer overlaps
"artist_is_only_songwriter", "artist_is_only_producer", "songwriter_is_a_producer",
# Timing
"instrumental_length_sec", "intro_length_sec",
# Lyrical content
"lyrical_topic", "lyrical_narrative",
# External content
"written_for_a_play", "written_for_a_film", "written_for_a_t_v_show", "eurovision_entry", "topped_the

billboard_manual_dropped <- billboard_model_final |>
  select(-all_of(columns_to_drop))

```

Drop columns with nearly zero variance Near-zero variance predictors were identified and removed using `nearZeroVar()` from the `caret` package. A predictor was flagged for removal if it met either of two criteria: its most common value appeared at least 19 times more frequently than the second most common value (`freqCut = 95/5`), or fewer than 5% of its values were unique (`uniqueCut = 5`).

```

# Identify and remove near-zero variance predictors
nzv_cols <- nearZeroVar(billboard_manual_dropped,
                        freqCut = 95/5,
                        uniqueCut = 5,
                        names = TRUE)

billboard_model_selected <- billboard_manual_dropped |>
  select(-all_of(nzv_cols))

```

Regression Model

Split the dataset into training and testing sets, with 705 of the observations allocated to the training set and 30% to the testing set.

```

# Train/test split
billboard_split <- initial_split(billboard_model_selected, prop = 0.7)
train_data <- training(billboard_split)
test_data <- testing(billboard_split)

```

A linear regression workflow was constructed using a preprocessing recipe that dummy-encoded categorical variables, removed near-zero variance and collinear predictors, and standardized numeric predictors. The model was fit using ordinary least squares on the training data.

```

lm_recipe <- recipe(log_weeks_at_number_one ~ ., data = train_data) |>
  step_dummy(all_nominal_predictors(), one_hot = FALSE) |>
  step_nzv(all_predictors()) |>
  step_normalize(all_numeric_predictors()) |>
  step_lincomb(all_numeric_predictors())

lm_spec <- linear_reg() |>
  set_engine("lm") |>
  set_mode("regression")

lm_workflow <- workflow() |>
  add_recipe(lm_recipe) |>

```



```
add_model(lm_spec)
```

The linear regression workflow was evaluated using repeated 5-fold cross-validation, stratified by `log_weeks_at_number_one`, with RMSE, R^2 , and MAE used as performance metrics.

```
set.seed(123)
cv_folds <- vfold_cv(
  train_data,
  v      = 5,
  repeats = 3,
  strata = log_weeks_at_number_one
)

cv_results <- fit_resamples(
  lm_workflow,
  resamples = cv_folds,
  metrics    = metric_set(rmse, rsq, mae),
  control    = control_resamples(save_pred = TRUE)
)

# Cross-validation summary
cv_metrics <- collect_metrics(cv_results)
print(cv_metrics)
```

```
## # A tibble: 3 x 6
##   .metric .estimator  mean     n std_err .config
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1 mae     standard    0.415     15 0.00599 pre0_mod0_post0
## 2 rmse    standard    0.505     15 0.00674 pre0_mod0_post0
## 3 rsq     standard    0.0626    15 0.0113  pre0_mod0_post0
```

Fit the model to the training data

```
final_fit <- fit(lm_workflow, data = train_data)
```

The fitted model was evaluated on the test set by comparing predicted and observed values using RMSE, R^2 , and MAE.

```
test_predictions <- predict(final_fit, new_data = test_data) |>
  bind_cols(test_data |> select(log_weeks_at_number_one)) |>
  rename(
    predicted = .pred,
    actual    = log_weeks_at_number_one
  )

test_metrics <- test_predictions |>
  metrics(truth = actual, estimate = predicted)

print(test_metrics)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>      <dbl>
## 1 rmse    standard    0.502
## 2 rsq     standard    0.0518
## 3 mae     standard    0.395
```

Results

Visualization Actual version predicted weeks at number one on the test set, shown on the original scale, with the red dashed line indicating perfect prediction.

```
rsq_label <- paste0(
  "R_squared = ",
  round(
    test_predictions |>
      metrics(truth = actual, estimate = predicted) |>
      filter(.metric == "rsq") |>
      pull(.estimate),
    3
  )
)

p_actual_vs_predicted <- test_predictions |>
  mutate(
    actual_weeks = expm1(actual),
    predicted_weeks = expm1(predicted)
  ) |>
  ggplot(aes(x = actual_weeks, y = predicted_weeks)) +
  geom_point(alpha = 0.4, colour = "steelblue", size = 2) +
  geom_abline(
    slope = 1,
    intercept = 0,
    linetype = "dashed",
    colour = "firebrick",
    linewidth = 0.8
  ) +
  annotate(
    "text",
    x = 12,
    y = 2,
    label = rsq_label,
    colour = "firebrick",
    size = 4
  ) +
  labs(
    title = "Actual vs Predicted Weeks at #1 (Test Set)",
    x = "Actual Weeks at #1",
    y = "Predicted Weeks at #1"
  ) +
  theme_minimal()

p_actual_vs_predicted
```

Top 15 predictors by standardized coefficient magnitude, with color indicating positive or negative effect on the response.

```
# Extract coefficients safely using broom::tidy explicitly
coef_data <- broom::tidy(extract_fit_engine(final_fit)) |>
  filter(term != "(Intercept)") |>
  arrange(desc(abs(estimate))) |>
  slice_head(n = 15) |>
  mutate(
```

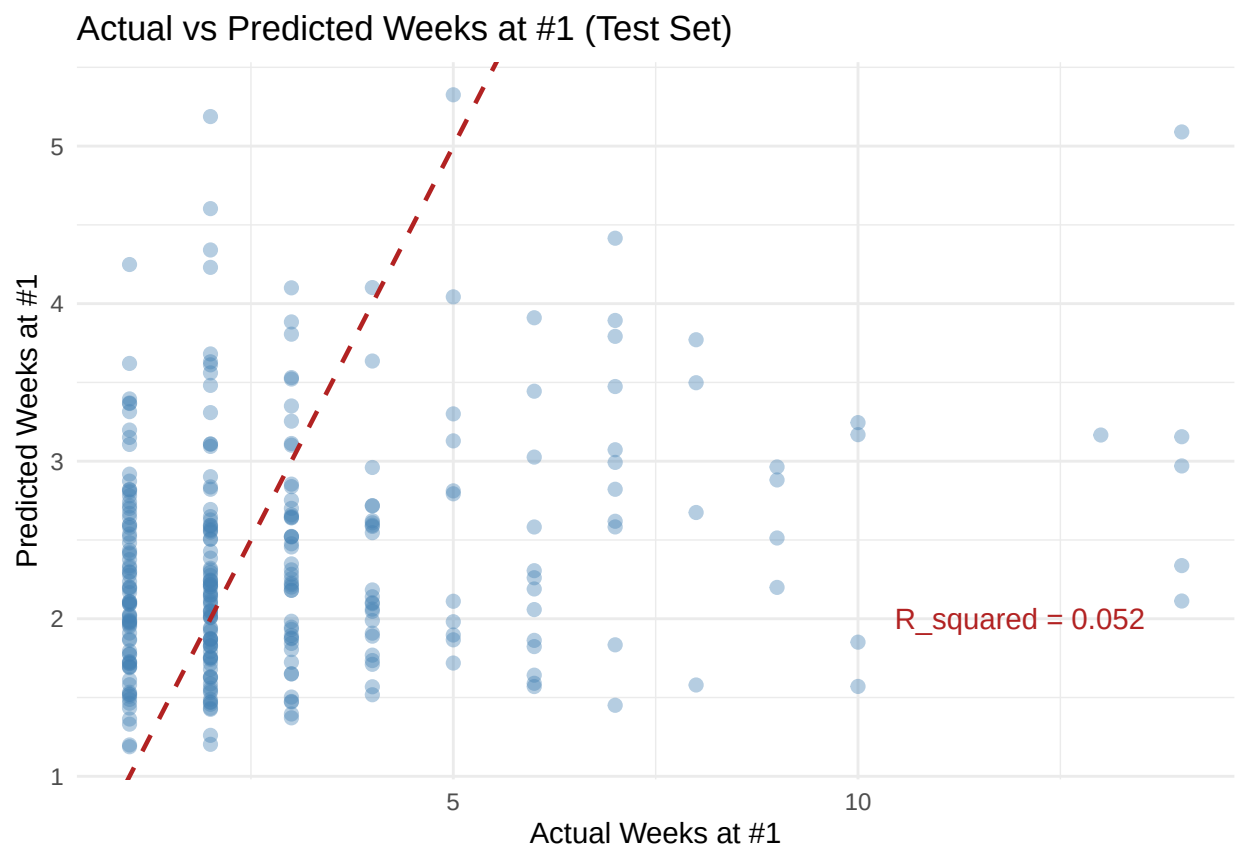


Figure 4: Actual vs predicted weeks at number one on the test set (original scale).

```

    direction = if_else(estimate > 0, "Positive", "Negative"),
    term       = str_replace_all(term, "_", " ") |> str_to_title()
  )

p_coefficients <- ggplot(
  coef_data,
  aes(x = reorder(term, abs(estimate)), y = estimate, fill = direction)
) +
  geom_col(show.legend = TRUE) +
  coord_flip() +
  scale_fill_manual(
    values = c("Positive" = "steelblue", "Negative" = "firebrick"),
    name   = "Effect on\nlog(weeks + 1)"
  ) +
  labs(
    title = "Top 15 Predictors by Coefficient Magnitude",
    x     = NULL,
    y     = "Standardised Coefficient"
  ) +
  theme_minimal() +
  theme(legend.position = "right")

p_coefficients

```

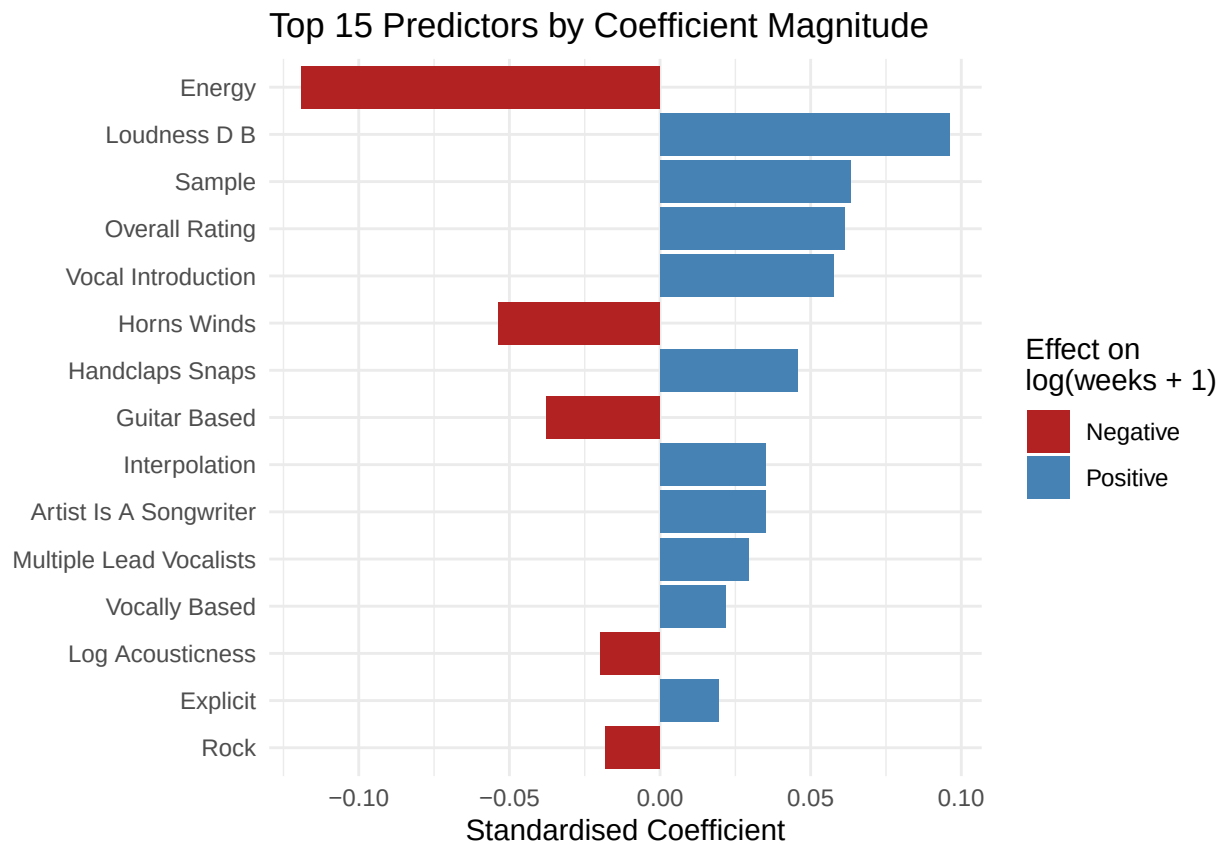


Figure 5: Top 15 predictors by standardised coefficient magnitude.

```
coef_data
```

```
## # A tibble: 15 x 6
##   term                estimate std.error statistic  p.value direction
##   <chr>              <dbl>    <dbl>    <dbl>    <dbl> <chr>
## 1 Energy             -0.119    0.0344   -3.45  0.000590 Negative
## 2 Loudness D B        0.0960    0.0287    3.35  0.000857 Positive
## 3 Sample              0.0634    0.0377    1.68  0.0929    Positive
## 4 Overall Rating      0.0613    0.0203    3.02  0.00260    Positive
## 5 Vocal Introduction  0.0576    0.0208    2.77  0.00573    Positive
## 6 Horns Winds         -0.0535   0.0221   -2.42  0.0158    Negative
## 7 Handclaps Snaps     0.0456    0.0194    2.35  0.0191    Positive
## 8 Guitar Based        -0.0376   0.0231   -1.63  0.104     Negative
## 9 Interpolation        0.0350    0.0432    0.810 0.418     Positive
## 10 Artist Is A Songwriter 0.0349    0.0252    1.38  0.167     Positive
## 11 Multiple Lead Vocalists 0.0292    0.0205    1.43  0.154     Positive
## 12 Vocally Based       0.0219    0.0199    1.10  0.271     Positive
## 13 Log Acousticness    -0.0198   0.0236   -0.840 0.401     Negative
## 14 Explicit            0.0193    0.0245    0.787 0.432     Positive
## 15 Rock                -0.0182   0.0404   -0.451 0.652     Negative
```

Result Interpretation The linear regression model demonstrated weak predictive performance, with cross-validation yielding at R^2 of 0.063 (RMSE = 0.505, MAE = 0.415) and test set evaluation confirming this with an R^2 of 0.052 (RMSE = 0.502, MAE = 0.395). The close alignment between cross-validation and test metrics indicates the model generalizes consistently, though its overall explanatory power remains low. As shown in Figure 4, the model can not properly predict the values of weeks that song remains at 1. Among the most 15 influential predictors, **energy** has the strongest effect at ($\beta = -0.119, p < 0.001$), suggesting that lower-energy songs tend to stay longer at #1, followed by **loudness** ($\beta = 0.096, p < 0.001$) and **overall_rating** ($\beta = 0.061, p = 0.003$). However, the majority of predictors, including **sample**, **interpolation**, **explicit**, and **Rock**, were not statistically significant at the 0.05 level, indicating that most structural and genre features carry little predictive signal once audio features are accounted for.

Discussion

The objective of our analysis was to use a linear regression model to predict the number of weeks a song would remain at the #1 spot of the Billboard Hot 100 song rankings (**weeks_at_number_one**) given open source data. Unnecessary features and features risking data-leakage into the predictive model were removed, and the skewed numerical features were transformed. One Hot Encoding was applied on categorical features, including genre and musical key; to predict on the data. Following this preprocessing, the linear regression model was trained the predictions were compared to the ground truth of weeks at the #1 spot. We found that in spite of its limited predictive power, that the model generalizes well onto unseen data due to the small difference between training and testing error, and that the top 3 statistically significant features involved in prediction were Energy, Loudness, and overall rating. From the coefficient values, we can conclude that weeks a song remains at the number one position is most associated with lower energy, higher loudness, and higher overall popularity.

Previous literature has also investigated song popularity with various models including KNN, linear regression, Random forest (and other tree-based models), and neural networks. Although these models have varying levels of predictive performance, song popularity is documented as a challenge to predict due to the nuanced level of the problem. Factors including individual preferences, globalization of music through various streaming platforms, and global events contribute to the challenge of measuring and influencing song popularity. (Terroso-Saenz, Soto, and Muñoz 2023) Further, global music trends and top performing artists have shifted since the introduction of streaming platforms, and song lengths have decreased to adjust to preferences over time. (Kim 2021; Terroso-Saenz, Soto, and Muñoz 2023) Since our analysis spans multiple

decades, world events, and to the adjustment of streaming platforms, these external factors could be influencing our R^2 value and error metrics. This means that songs with popular characteristic features leading to a high number of weeks at the top position from the 1950's or 60's, may not generalize well to features from popular songs in the 21st century.

A specific example of classifier implementation was conducted by Kim in 2021. This study, similar to the present study, investigated the impact of various features contributing to song popularity from 2010-2019 using Spotify data, and analyzed the performance through the RMSE of logistic regression, KNN, and Random Forest models. The lowest RMSE score was associated with the logistic regression model, followed by KNN and Random forest. (Kim 2021)

Overall, this regression model provides good predictions given the data, historical trends in music forecasting, and questions being investigated; in spite of its weak predictive patterns. It adds to previous literature, by implementing different features and conducting a longitudinal study. Based on our findings, improvements can be made through training separate models for the data into the 20th vs the 21st century, or by decade, to improve performance; while retaining its longitudinal nature.

References

- Billboard Staff. 2011. "About Us: Billboard." <https://www.billboard.com/music/music-news/about-us-467859/>.
- Data Science Learning Community. 2025. "Tidy Tuesday: A Weekly Social Data Project." <https://github.com/rfordatascience/tidytuesday/tree/main/data/2025/2025-08-26>.
- Kim, J. 2021. "Music Popularity Prediction Through Data Analysis of Music's Characteristics." *International Journal of Science Technology and Society* 9 (5): 239. <https://doi.org/10.11648/j.ijsts.20210905.16>.
- Terroso-Saenz, F., J. Soto, and A Muñoz. 2023. "Evolution of Global Music Trends: An Exploratory and Predictive Approach Based on Spotify Data." *Entertainment Computing* 44. <https://doi.org/10.1016/j.entcom.2022.100536>.